

Lab 05 Tasks

General Requirements

For every task:

- Use **Python, OpenCV and NumPy**.
- Display the **original image and segmentation result**.
- Do not use a prebuilt segmentation library or deep-learning segmentation model.
- Record the parameters you selected and briefly justify them.
- Save the final output of every task.

Task 1 – Automated Inspection of a Document Under Uneven Lighting

You are developing an automated document-scanning system for a university archive. A document was photographed using a mobile phone, but the lighting is not uniform: one side of the page is brighter than the other. The system must separate the printed text from the background.

A single global threshold may produce missing characters in dark areas or unwanted regions in bright areas.

Your Task

Given the document image:

Perform the following:

1. Load the image in grayscale.
2. Apply **global thresholding** using at least **three different threshold values**.
3. Compare the resulting binary masks.
4. Apply **adaptive thresholding** to the same image.
5. Compare the adaptive result against your best global-threshold result.
6. Identify which regions of the document cause global thresholding to fail.

7. Select the parameter combination that produces the most usable document mask.

Required Output

Create a comparison containing:

Original → Global T1 → Global T2 → Global T3 → Adaptive

Under the final result, write:

Why does a single threshold struggle when illumination changes across the image?

Task 2 – Choosing the Correct Adaptive Threshold Strategy

The archive team reports that simply switching to adaptive thresholding is not enough. Some versions of the document produce broken characters, while others create excessive black noise around the letters.

You must determine how the **local neighbourhood size** and **constant C** affect segmentation.

Your Task

Talha Shahid

Use the same unevenly illuminated document from Task 1.

Test:

- At least **three different blockSize values**
- At least **three different C values**
- Both adaptive methods discussed in the manual:
 - Mean-based
 - Gaussian-based

Create a parameter comparison.

Required Analysis

Determine:

1. What happens when the neighbourhood is too small?
2. What happens when the neighbourhood is too large?
3. What happens when C is increased?

4. Which combination produces the cleanest text segmentation?

5. Which adaptive method performs better on this image?

Your final output should show the **original plus at least six different adaptive segmentation results**.

Task 3 – Quality-Control System Using Otsu's Thresholding

A manufacturing company photographs a set of objects against a relatively consistent background. The intensity distribution is expected to contain two major classes:

object pixels + background pixels

The engineer does not want to manually determine the threshold for every image.

Your Task

Use a grayscale object image such as the coins image associated with OpenCV's watershed example:

Perform the following:

1. Convert the image to grayscale.
2. Generate a histogram of the grayscale image.
3. Apply **Otsu's thresholding**.
4. Record the automatically selected threshold.
5. Repeat the experiment after changing the image contrast or preprocessing slightly.
6. Compare the automatically selected thresholds.

Required Output

Show:

Original → Histogram → Otsu Binary Mask

Then answer:

Why is Otsu useful when the programmer does not know the correct threshold beforehand?

Your implementation must use Otsu's thresholding rather than manually choosing the threshold, as specified in the manual.

Task 4 – Sorting Objects by Color

A robotic sorting system receives an image containing objects of different colors. The robot is interested only in one particular colored object and needs a binary mask identifying that object.

Intensity alone cannot reliably distinguish the desired object from the background.

Your Task

Use a color image containing a clearly distinguishable colored object, such as the yellow-car example:

Build a **color-based segmentation** system.

Your program must:

1. Load the image in color.
2. Convert it to **HSV**.
3. Define a lower and upper color range.
4. Generate a binary mask using the color range.
5. Experiment with the lower and upper limits.
6. Extract the selected color from the original image using your mask.

Challenge Requirement

Create **two masks**:

- A mask that is too restrictive
- A final mask that captures the desired object more completely

Then explain why the restrictive mask fails.

The manual specifically presents HSV-based color thresholding using lower and upper bounds.

Task 5 – Detecting the Boundary of a Manufactured Part

A factory camera must identify the boundary of a mechanical component before its shape is analyzed. The system does not need to keep the interior of the object; it needs to highlight rapid changes in intensity corresponding to boundaries.

The camera images contain both strong and weak edges.

Your Task

Choose a suitable object image and perform **Canny edge detection**.

Test at least:

- One low/high threshold pair
- A second low/high threshold pair
- A third low/high threshold pair

For example, investigate what happens when the high threshold is increased while the low threshold remains fixed.

Required Output

Produce:

Talha Shahid

Original → Edge Result 1 → Edge Result 2 → Edge Result 3

Then identify:

- Strong edges
- Weak edges
- Missing edges
- Unwanted edges

Explain how changing the two thresholds changes the final result.

Your implementation should use the two-threshold hysteresis concept described in the manual.

Task 6 – Segmenting a Region Inside a Medical Image

A medical imaging researcher wants to isolate a contiguous region of similar intensity from a brain MRI. The target region is not easily separated from the entire image using a single global threshold because several areas have similar intensities.

Instead of classifying every pixel independently, the researcher wants to start from a known location and allow the region to grow through neighbouring pixels that satisfy an intensity condition.

Use a brain MRI such as:

Your Task

Implement **region growing**.

Your program must:

1. Load a grayscale image.
2. Select a seed point.
3. Grow the region from that point.
4. Use an intensity difference threshold to decide whether neighbouring pixels belong to the region.
5. Produce a binary segmentation mask.

Parameter Study

Repeat the experiment with at least **three different region-growing thresholds**.

Also test at least **two different seed locations**.

Required Analysis

Explain:

Why can changing the seed point significantly change the final segmented region?

Task 7 – Separating Touching Coins in an Automated Counting System

A manufacturing system counts circular components placed on a tray. Unfortunately, several components are touching each other.

A simple threshold produces one connected white region containing multiple touching objects. Counting connected regions directly therefore produces an incorrect count.

Your Task

Develop a **marker-based watershed segmentation pipeline**.

Your implementation must follow the stages presented in the manual:

1. Preprocessing
2. Thresholding
3. Noise removal
4. Sure-background generation
5. Distance transform
6. Sure-foreground generation
7. Unknown-region generation
8. Marker labeling
9. Watershed transformation
10. Boundary visualization

Required Output

Display:

Original → Threshold → Sure Background → Distance Transform → Sure Foreground → Unknown Region → Final Watershed

Finally, visually inspect whether touching coins have been separated.

Task 8 – Tuning Watershed for Difficult Object Separation

The coin-counting system from Task 7 works reasonably well, but some objects are merged while others are incorrectly split.

The problem appears to be related to how much of each object is considered **sure foreground**.

Your Task

Modify your Task 7 program and investigate the threshold applied to the **distance transform**.

Perform at least **four experiments** using different distance-transform thresholds.

For every experiment record:

- Distance-transform threshold
- Number of detected foreground markers
- Number of separated regions
- Visual quality of segmentation

Required Output

Create a table such as:

Experiment	Distance Threshold	Foreground Regions	Observed Result
------------	--------------------	--------------------	-----------------

1	---	---	---
2	---	---	---
3	---	---	---
4	---	---	---

Then identify the parameter range that gives the most meaningful separation.

Do not replace watershed with another segmentation method. This task is specifically about understanding the marker and distance-transform stages from the manual.

Task 9 – Segmenting an Image Using K-Means

A wildlife researcher receives a color photograph and wants to simplify it into a small number of visually meaningful regions.

A single threshold is not appropriate because many different colors and intensity values occur in the image.

The researcher decides to cluster pixels according to their color similarity.

Use a complex color image such as:

Your Task

Implement **K-Means-based image segmentation**.

Your program should:

1. Load a color image.
2. Represent the image as a set of pixel feature vectors.
3. Convert the pixel data to the appropriate numeric type.
4. Perform K-Means clustering.
5. Reconstruct the segmented image using cluster centers.

Run the experiment for:

K = 2, 4 and 6

Required Output

Display:

Original → K=2 → K=4 → K=6

For each K, record:

- Number of clusters
- Visual difference from the original
- Amount of color simplification
- Major regions represented by the clusters

The procedure should follow the K-Means workflow in the manual: reshape pixels, convert to float32, cluster, obtain centers, and reconstruct the image.

Task 10 – Design a Segmentation System for an Unknown Image

You have joined a computer-vision development team. Your team receives an image but is not told which segmentation method should be used.

Your task is to act as the segmentation engineer.

Choose an image containing a reasonably challenging segmentation problem.

Examples include:

- unevenly illuminated document
- colored object
- touching objects
- grayscale object/background
- complex natural scene

You may use one of the images from the previous tasks.

Your Task

Implement **three different segmentation approaches**:

You must choose from:

- Global thresholding
- Adaptive thresholding
- Otsu's thresholding
- Color-based thresholding
- Canny edge detection
- Region growing
- Watershed
- K-Means

Additional Requirement

Do not simply run the methods and display their outputs.

For every selected method, identify:

1. What assumption the method makes about the image
2. What region/object it successfully identifies

3. What part of the image it incorrectly segments
4. Which parameter has the greatest effect on the result

Final Deliverable

Create one final comparison:

Original | Method 1 | Method 2 | Method 3

Then write a short technical conclusion:

Which segmentation approach produced the most meaningful regions for this particular image, and what property of the image made that method suitable?

Your conclusion should be based on the **observed segmentation results**, not simply on which algorithm you already know.

Talha Shahid